

```

Hackintosh:~# netperf -s
Starting netserver with host "127.0.0.1" port "1234" and family AF_INET
Hackintosh:~# netperf -c 127.0.0.1 -p 1234
HIGHEST TCP STREAM TEST from 6.0-6.0 (0.0-0.0) port 0 AF_INET to localhost (1) port 0 AF_INET
Recv Send
Hosts: 127.0.0.1 127.0.0.1
Size: 16384 16384
Type: TCP_STREAM
Time: 15.00
Throughput: 10739.32

```

Figure 6: *netperf* running as a server and client on the same system

```

Hackintosh:~# iperf -s -p 1234
server listening on 1234
Accepted connection from 127.0.0.1 port 57346
[ ID] local 127.0.0.1 port 1234 connected to 127.0.0.1 port 57346
[ ID] Interval Transfer Bandwidth
[ 0] 0.00-1.00 sec 4.35 Gbytes 37.4 Gbits/sec
[ 1] 1.00-2.00 sec 3.92 Gbytes 33.7 Gbits/sec
[ 2] 2.00-3.00 sec 3.53 Gbytes 30.4 Gbits/sec
[ 3] 3.00-4.00 sec 3.39 Gbytes 29.1 Gbits/sec
[ 4] 4.00-5.00 sec 3.45 Gbytes 29.7 Gbits/sec
[ 5] 5.00-6.00 sec 3.42 Gbytes 29.4 Gbits/sec
[ 6] 6.00-7.00 sec 3.46 Gbytes 29.7 Gbits/sec
[ 7] 7.00-8.00 sec 3.49 Gbytes 29.9 Gbits/sec
[ 8] 8.00-9.00 sec 3.41 Gbytes 29.3 Gbits/sec
[ 9] 9.00-10.00 sec 3.48 Gbytes 29.8 Gbits/sec
[10] 10.00-10.04 sec 132 Mbytes 23.8 Gbits/sec
[ ID] Interval Transfer Bandwidth Retx sender
[ 0] 0.00-10.04 sec 35.9 Gbytes 30.8 Gbits/sec 0 receiver
[ 1] 0.00-10.04 sec 35.9 Gbytes 30.8 Gbits/sec 0 receiver

```

Figure 7: *iperf* running as server

of your system as there are no generic tweaks for all systems. Another important point to remember is that you must develop a good understanding of the entire system. The only part of a system that should ever be tuned is the bottleneck. Improving the performance of any other component other than the bottleneck will result in no visible performance gains to the end user. But the focus of this article is on the general, potential bottlenecks you might come across; even on healthy systems, these tweaks may give improved performance.

One obvious situation to tune your network for is when you start to drop received packets or transmit/receive the message ‘error has occurred’. This can happen when your packet holding structures in the NIC, device driver, kernel or network layer are not able to handle all the received packets or are not able to process these fast enough. The bottleneck in this case can be related to any of the following:

- 1) NIC
- 2) Soft interrupt issued by a device driver
- 3) Kernel buffer
- 4) The network layer (IP, TCP or UDP)

It’s better to find the exact bottleneck from among these options and then tune only that area. Or you can apply the methods given below, one by one, and then find which changes improve performance.

To find out the packet drop/error details, you can use the *ip* utility.

More specific results can be viewed by using *ethtool*, as follows:

```
ethtool -s eth0
```

Tuning the network adapter (NIC)

Jumbo frames: A single frame can carry a maximum of 1500 bytes by default as shown by the *ip addr* command.

```

Hackintosh:~# iperf -c 127.0.0.1 -p 1234
Connecting to host 127.0.0.1 port 1234
[ ID] local 127.0.0.1 port 57346 connected to 127.0.0.1 port 1234
[ ID] Interval Transfer Bandwidth Retx Cwnd
[ 0] 0.00-1.00 sec 4.50 Gbytes 38.7 Gbits/sec 0 1.25 Mbytes
[ 1] 1.00-2.00 sec 3.98 Gbytes 34.2 Gbits/sec 0 1.25 Mbytes
[ 2] 2.00-3.00 sec 3.66 Gbytes 29.7 Gbits/sec 0 1.04 Mbytes
[ 3] 3.00-4.00 sec 3.29 Gbytes 28.1 Gbits/sec 0 1.04 Mbytes
[ 4] 4.00-5.00 sec 3.45 Gbytes 29.6 Gbits/sec 0 1.04 Mbytes
[ 5] 5.00-6.00 sec 3.42 Gbytes 29.4 Gbits/sec 0 1.04 Mbytes
[ 6] 6.00-7.00 sec 3.46 Gbytes 29.7 Gbits/sec 0 1.04 Mbytes
[ 7] 7.00-8.00 sec 3.48 Gbytes 29.9 Gbits/sec 0 1.04 Mbytes
[ 8] 8.00-9.00 sec 3.40 Gbytes 29.3 Gbits/sec 0 3.31 Mbytes
[ 9] 9.00-10.00 sec 3.39 Gbytes 29.1 Gbits/sec 0 3.31 Mbytes
[ ID] Interval Transfer Bandwidth Retx sender
[ 0] 0.00-10.00 sec 35.9 Gbytes 30.8 Gbits/sec 0 receiver
[ 1] 0.00-10.00 sec 35.9 Gbytes 30.8 Gbits/sec 0 receiver
iperf Done.

```

Figure 8: *iperf* running as a client

```

Hackintosh:~# ip -s link show eth0
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc pfifo_fast state DOWN mode
DEFAULT group default qlen 1000
link/ether 28:d2:4a:c1:7b:cc brd ff:ff:ff:ff:ff:ff
RX: bytes packets errors dropped overrun mcast
0 0 0 0 0 0
TX: bytes packets errors dropped carrier collsns
0 0 0 0 0 0

```

Figure 9: *ip* result for a healthy system with no packet drops

Here, for *eth0*, which is the default network interface for the Ethernet, the MTU (maximum transfer unit) value is set to 1500. This value is defined for a single slot. Ethernet connections of 10Gbps or more may need to increase the value to 9000. These large frames are called Jumbo frames. To get Ethernet to use Jumbo frames, you can use *ifconfig*, as follows:

```
ifconfig eth0 mtu 9000
```

Note: Setting the value to 9000 bytes doesn’t make all frame sizes that large, but this value will be only used depending on your Ethernet requirements.

Interrupt Coalescence (IC)

After the network adapter receives the packets, the device driver issues a single hard interrupt followed by soft interrupts (handled by NAPI). Interrupt Coalescence is the number of packets the network adapter receives before issuing a hard interrupt. Changing the value to cause interrupts fast can lead to lots of overhead and, hence, decrease performance, but having a high value may lead to packet loss. By default, your settings are in adaptive IC mode, which auto balances the value according to network traffic. But since the new kernels use NAPI, which makes fast interrupts much less costly (performance wise), you can disable this feature.

To check whether the adaptive mode is on or not, run the following command:

```
$ ethtool -c eth0
```

To disable it, use the following command:

```
$ ethtool -C eth0 adaptive-rx off
```